

Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Інститут комп'ютерних наук та інформаційних технологій
Кафедра автоматизованих систем управління



Звіт

до лабораторної роботи №4
з дисципліни:

“Паралельні обчислення і розподілені системи”

на тему:

“Мікросервіси. Декомпозиція моноліту”

Виконав: студент ОІ-31
Джебко Юрій

Прийняв:
Скорохода О. В.

Львів – 2026

Лабораторна робота №4

Мікросервіси. Декомпозиція моноліту

Мета роботи: забезпечити логічну ізоляцію відповідальності; незалежне розгортання сервісів; можливість горизонтального масштабування.

Хід роботи

1. Постановка задачі:

У поточній системі є 6 сутностей: Client, Shipment, Status, Warehouse, Route, Transport. У моноліті вони зв'язані жорсткими індексами на рівні однієї БД.

Для декомпозиції я виділив 3 незалежні сервіси:

Service A: Shipment & Tracking Service (Сервіс посилок та трекінгу)

- Межі відповідальності: Реєстрація клієнтів, створення посилок, генерація трек-номерів, ведення історії статусів.
- Сутності: Client, Shipment, Status.
- Порт: 8081
- БД: shipment_db

Service B: Warehouse Service (Сервіс складської логістики)

- Межі відповідальності: Облік складів, контроль місткості (capacity та currentLoad), переміщення вантажів між складами.
- Сутності: Warehouse.
- Порт: 8082
- БД: warehouse_db

Service C: Routing & Transport Service (Сервіс автопарку та маршрутів)

- Межі відповідальності: Управління транспортом, реєстрація маршрутів, алгоритм оптимізації доставки (optimize).
- Сутності: Transport, Route.
- Порт: 8083
- БД: routing_db

2. Проєктування:

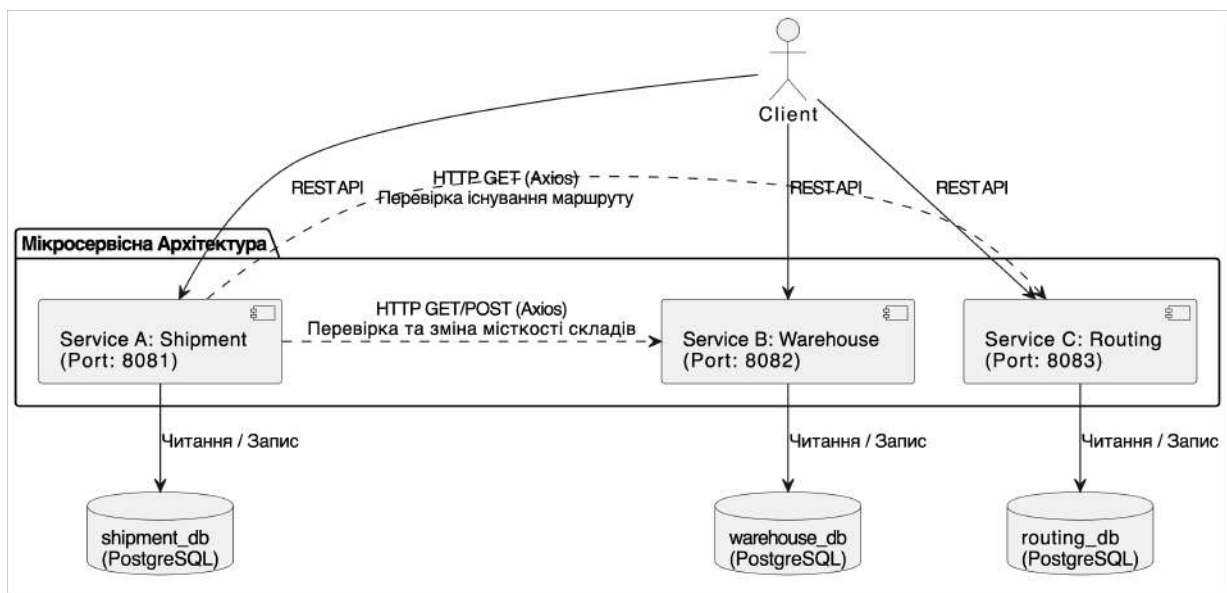


Рис. 1. Схема мікросервісної архітектури.

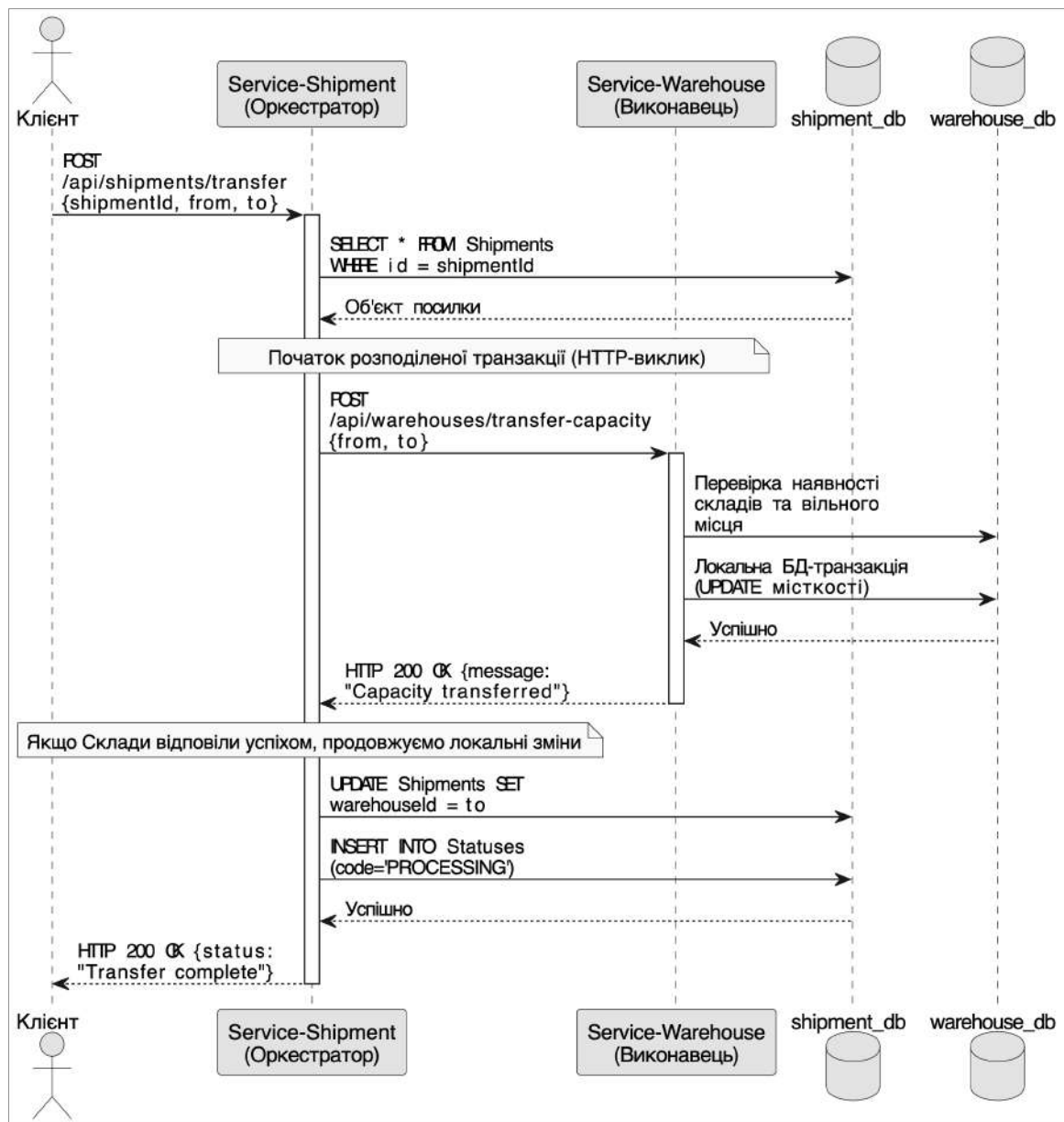


Рис. 2. Взаємодія між сервісами.

Порти та бази даних:

Назва мікросервісу	Порт	База даних (PostgreSQL)	Зона відповідальності (Сутності)
Service-Shipment	8081	shipment_db	Реєстрація клієнтів, облік посилок, трекінг статусів (Client, Shipment, Status)
Service-Warehouse	8082	warehouse_db	Управління складами, контроль місткості (Warehouse)
Service-Routing	8083	routing_db	Управління автопарком, прокладання маршрутів доставки (Transport, Route)

3. Реалізація:

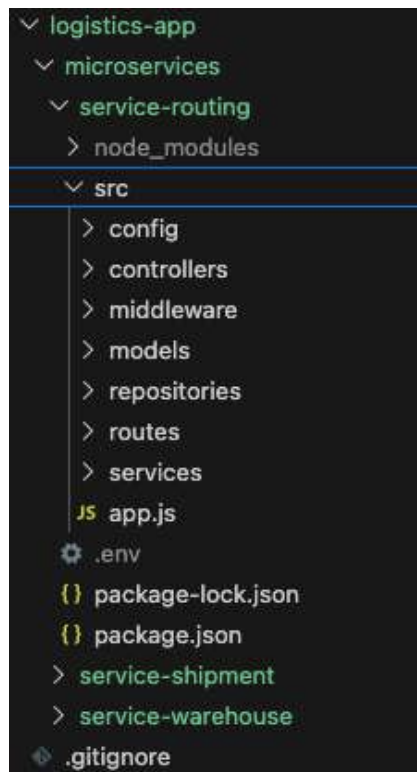


Рис. 3. Структура проєкту, розбитого на мікросервіси.

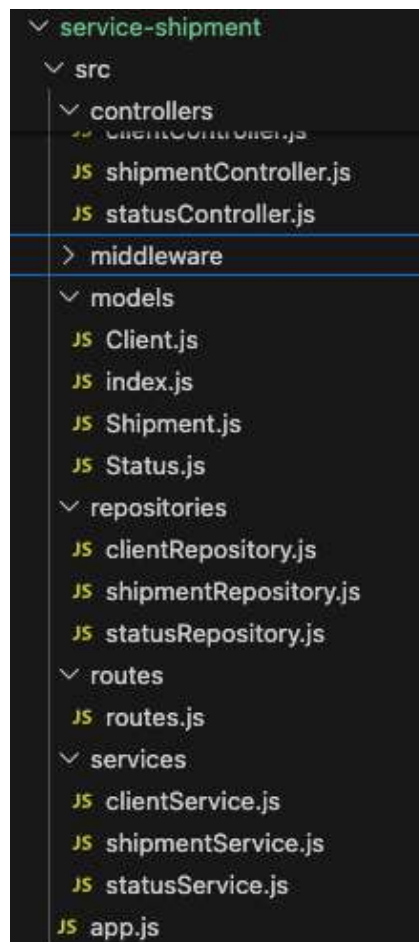


Рис. 4. Файлова структура для одного мікросервісу(service-shipment).

Приклад оновленого route.js для service-routing:

```
const router = require('express').Router();
const validate = require('../middleware/validate');

const routeCtrl = require('../controllers/routeController');
const transportCtrl = require('../controllers/transportController');

// --- Transports ---
router.get('/transports', transportCtrl.getAll);
router.get('/transports/available', transportCtrl.getAvailable);
router.get('/transports/:id', transportCtrl.getById);
router.post('/transports', transportCtrl.create);
router.put('/transports/:id', transportCtrl.update);
router.delete('/transports/:id', transportCtrl.remove);

// --- Routes ---
router.get('/routes', routeCtrl.getAll);
router.post('/routes/optimize', routeCtrl.optimize);
router.get('/routes/:id', routeCtrl.getById);
router.post('/routes', routeCtrl.create);
router.put('/routes/:id', routeCtrl.update);
router.delete('/routes/:id', routeCtrl.remove);

module.exports = router;
```

Приклад оновленого models/index.js для service-routing:

```
const sequelize = require('../config/database');
const Transport = require('./Transport');
const Route = require('./Route');

// Один Транспорт може мати багато Маршрутів
```

```

Transport.hasMany(Route, { foreignKey: 'transportId' });
Route.belongsTo(Transport, { foreignKey: 'transportId' });

module.exports = {
  sequelize,
  Transport,
  Route
};

```

Приклад оновленого сервісу shipmentService.js для service-shipment із взаємодією з іншим мікросервісом:

```

const repo = require('../repositories/shipmentRepository');
const clientRepo = require('../repositories/clientRepository');
const statusRepo = require('../repositories/statusRepository');
const axios = require('axios'); // Додали axios для мережових запитів

module.exports = {
  getAll: async () => {
    const shipments = await repo.findAll();
    return await Promise.all(shipments.map(async s => {
      const plainS = s.get ? s.get({ plain: true }) : s;
      return {
        ...plainS,
        currentStatus: await statusRepo.findLatestByShipmentId(plainS.id),
      };
    }));
  },

  getById: async (id) => {
    const shipment = await repo.findById(id);
    if (!shipment) throw { status: 404, message: `Shipment with id=${id} not found` };
    const plainS = shipment.get ? shipment.get({ plain: true }) : shipment;

    return {
      ...plainS,
      currentStatus: await statusRepo.findLatestByShipmentId(id),
      statusHistory: await statusRepo.findByShipmentId(id),
    };
  },

  trackByNumber: async (trackingNumber) => {
    const shipment = await repo.findByTrackingNumber(trackingNumber);
    if (!shipment) {
      throw { status: 404, message: `Tracking number "${trackingNumber}" not found` };
    }
    const plainS = shipment.get ? shipment.get({ plain: true }) : shipment;

    return {
      ...plainS,
      currentStatus: await statusRepo.findLatestByShipmentId(plainS.id),
      statusHistory: await statusRepo.findByShipmentId(plainS.id),
    };
  },

  create: async (data) => {
    // Перевірка у власній БД
    if (!(await clientRepo.exists(data.clientId))) {
      throw { status: 400, message: `Client with id=${data.clientId} does not exist` };
    }
  }
};

```

```

// Міжсервісний виклик до Service Warehouse (порт 8082)
if (data.warehouseId) {
  try {
    const response = await axios.get(`http://localhost:8082/api/warehouses/${data.warehouseId}`);
    const warehouse = response.data;
    if (warehouse.currentLoad >= warehouse.capacity) {
      throw { status: 409, message: `Warehouse id=${data.warehouseId} is at full capacity` };
    }
  } catch (error) {
    if (error.response && error.response.status === 404) {
      throw { status: 400, message: `Warehouse with id=${data.warehouseId} does not exist` };
    }
    // Якщо сервіс складів лежить (вимкнений або впав)
    throw { status: 503, message: 'Warehouse service is currently unavailable' };
  }
}

// Міжсервісний виклик до Service Routing (порт 8083)
if (data.routeId) {
  try {
    await axios.get(`http://localhost:8083/api/routes/${data.routeId}`);
  } catch (error) {
    if (error.response && error.response.status === 404) {
      throw { status: 400, message: `Route with id=${data.routeId} does not exist` };
    }
    throw { status: 503, message: 'Routing service is currently unavailable' };
  }
}

const shipment = await repo.create(data);
const plainS = shipment.get ? shipment.get({ plain: true }) : shipment;

await statusRepo.create({ shipmentId: plainS.id, code: 'PENDING', note: 'Shipment created' });

return { ...plainS, currentStatus: await statusRepo.findLatestByShipmentId(plainS.id) };
},

update: async (id, data) => {
  if (!(await repo.exists(id))) throw { status: 404, message: `Shipment with id=${id} not found` };
  return await repo.update(id, data);
},

remove: async (id) => {
  if (!(await repo.exists(id))) throw { status: 404, message: `Shipment with id=${id} not found` };
  await repo.remove(id);
},

transferShipment: async (shipmentId, fromWarehouseId, toWarehouseId) => {
  // Перевіряємо, чи існує посилка у БД
  const shipment = await repo.findById(shipmentId);
  if (!shipment) throw { status: 404, message: `Shipment with id=${shipmentId} not found` };

  // HTTP-запит до Сервісу Складів, щоб він змінив місткість
  try {

```

```

        await axios.post('http://localhost:8082/api/warehouses/transfer-capacity',
{
    fromWarehouseId,
    toWarehouseId
  });
  } catch (error) {
    if (error.response) {
      throw { status: error.response.status, message:
error.response.data.message };
    }
    throw { status: 503, message: 'Warehouse service is currently unavailable'
};
  }

  // змінюємо склад у самій посилці
  await repo.update(shipmentId, { warehouseId: Number(toWarehouseId) });

  // Логуємо в історію статусів
  await statusRepo.create({
    shipmentId,
    code: 'PROCESSING',
    note: `Transferred from warehouse ${fromWarehouseId} to ${toWarehouseId}`
  });

  return {
    shipmentId: Number(shipmentId),
    movedFrom: fromWarehouseId,
    movedTo: toWarehouseId,
    status: 'Transfer complete'
  };
}
};

```

4. Тестування:

1. Тестування всередині сервісів

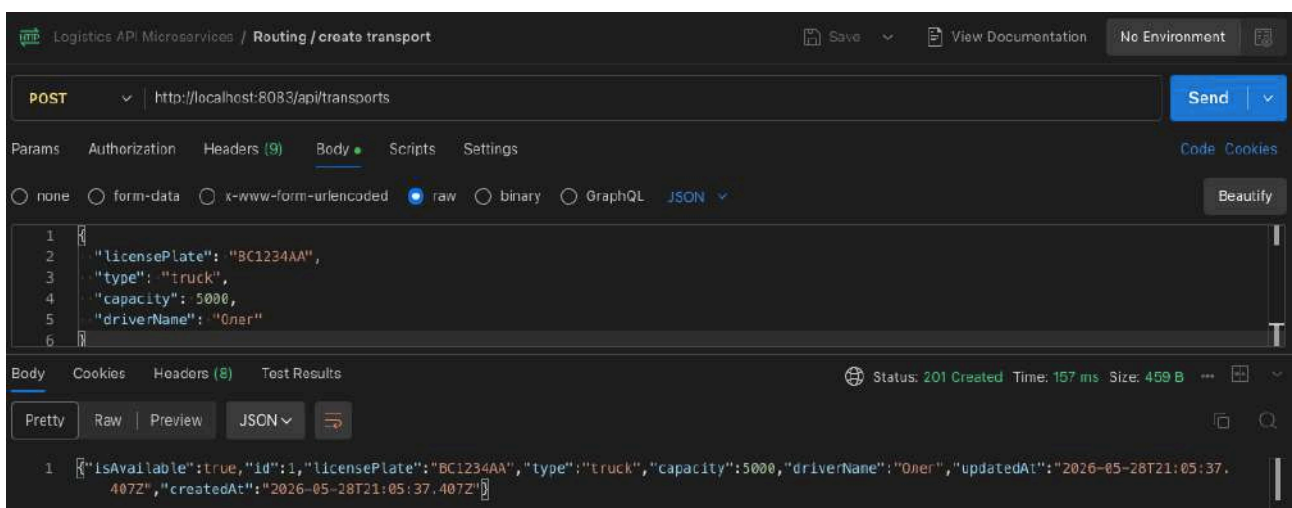


Рис. 5. Створення транспорту. POST-запит service-routing.

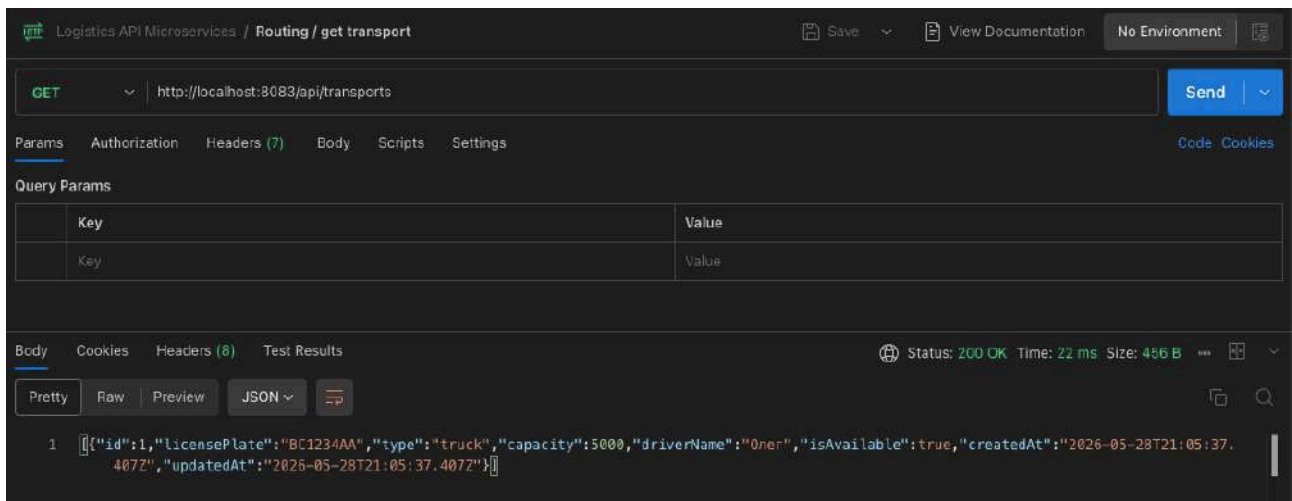


Рис. 6. Отримання списку транспорту. GET-запит сервісу service-routing.

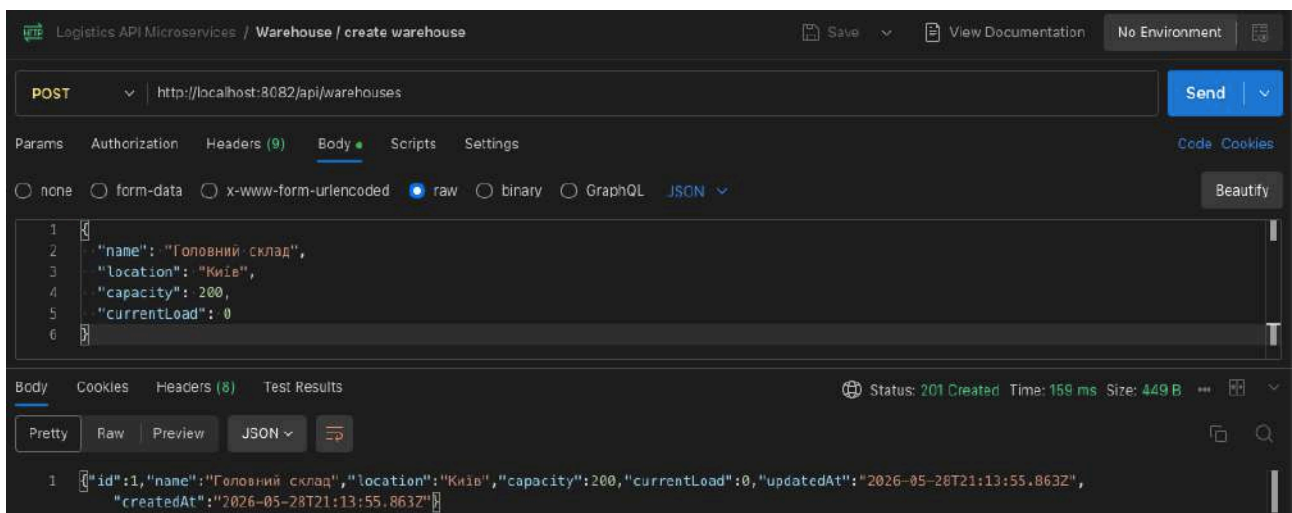


Рис. 7. Створення складу. POST-запит сервісу service-warehouse.

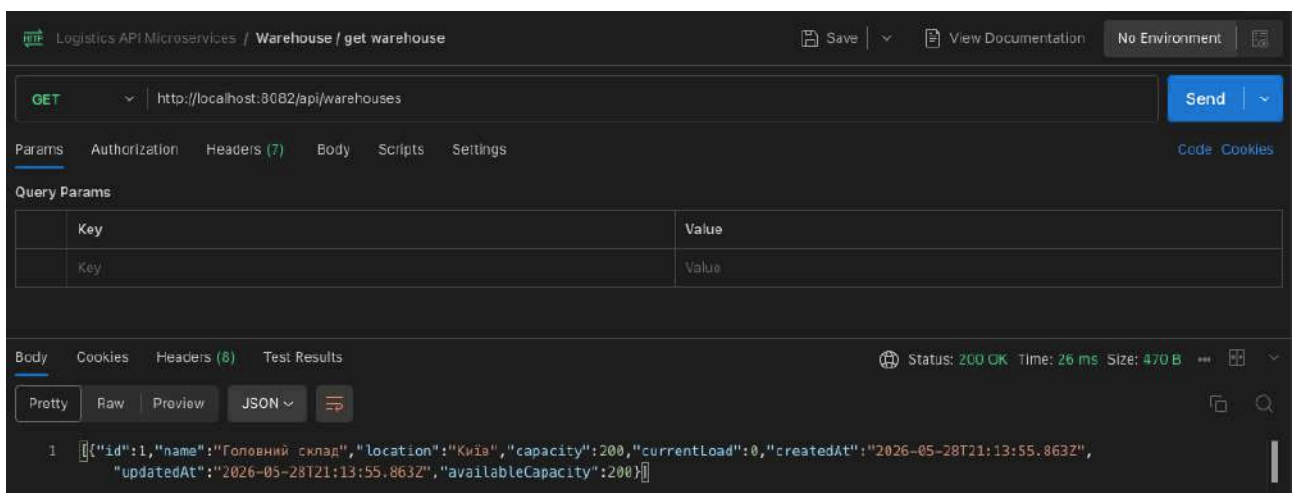


Рис. 8. Отримання списку складів. GET-запит сервісу service-warehouse.

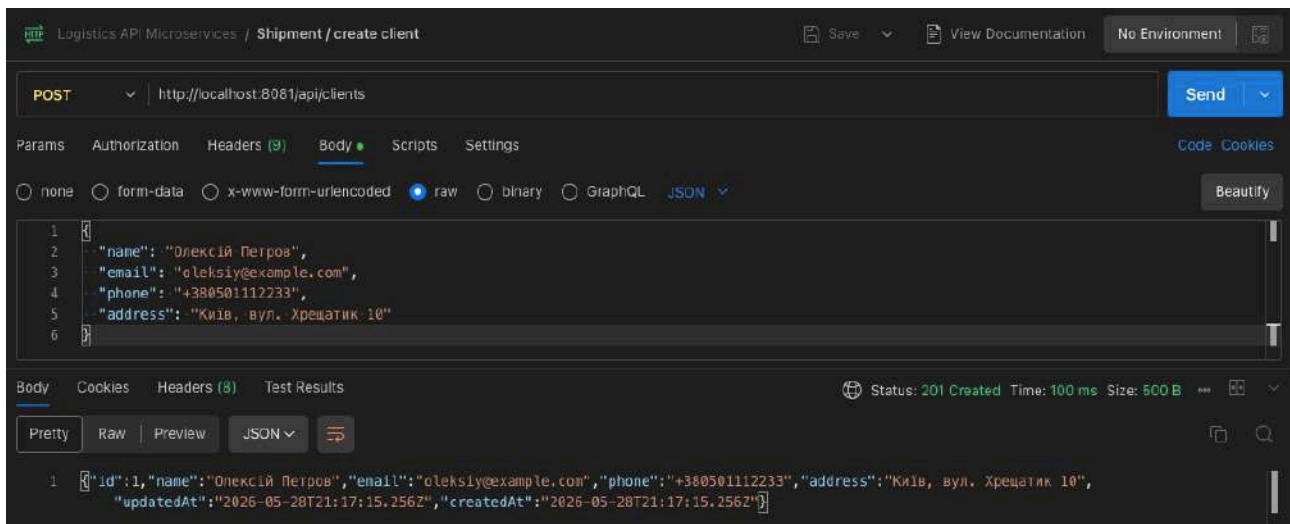


Рис. 9. Створення клієнта. POST-запит service-shipment.

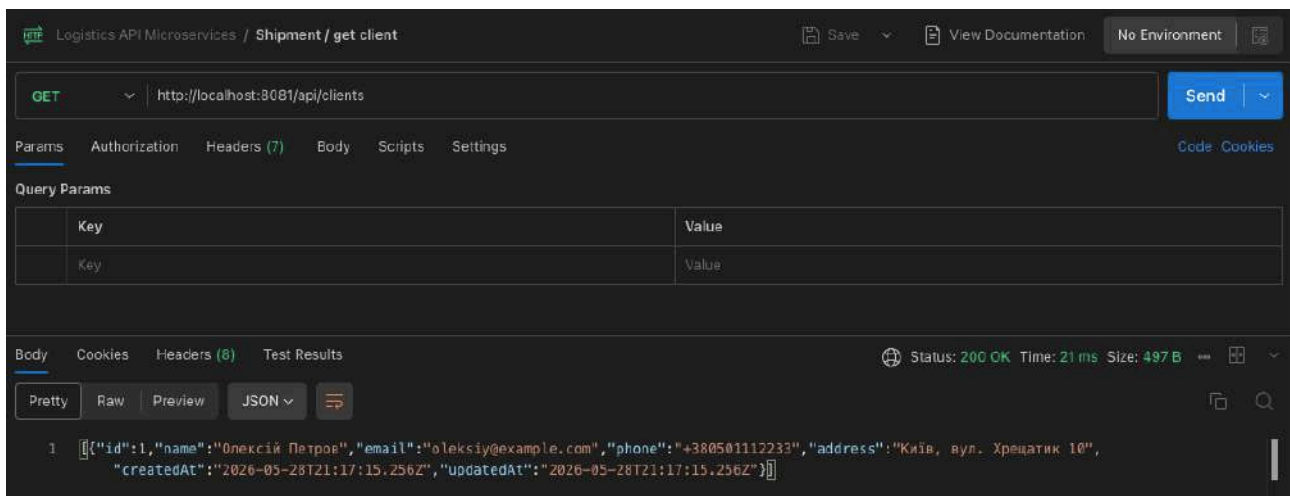


Рис. 10. Отримання списку клієнтів. GET-запит service-shipment.

2. Перевірка міжсервісної взаємодії

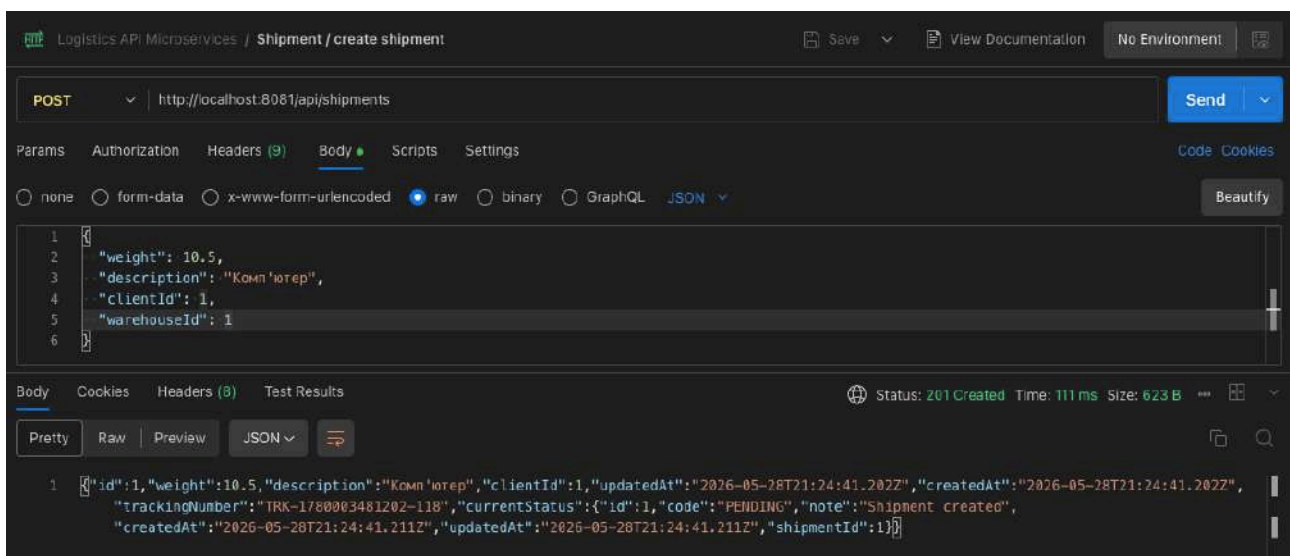


Рис. 11. Створення посилки. Сервіс 8081 робить непомітний GET-запит на 8082, перевіряє місце і створює посилку.

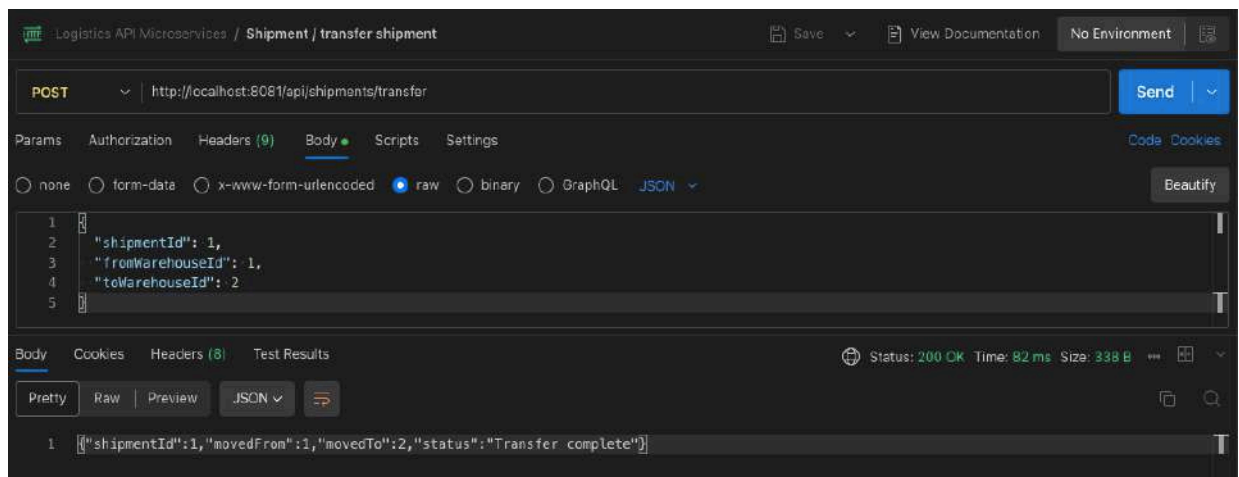


Рис. 12. Переміщення посилки. Переміщуємо створену посилку зі Складу 1 на Склад 2.

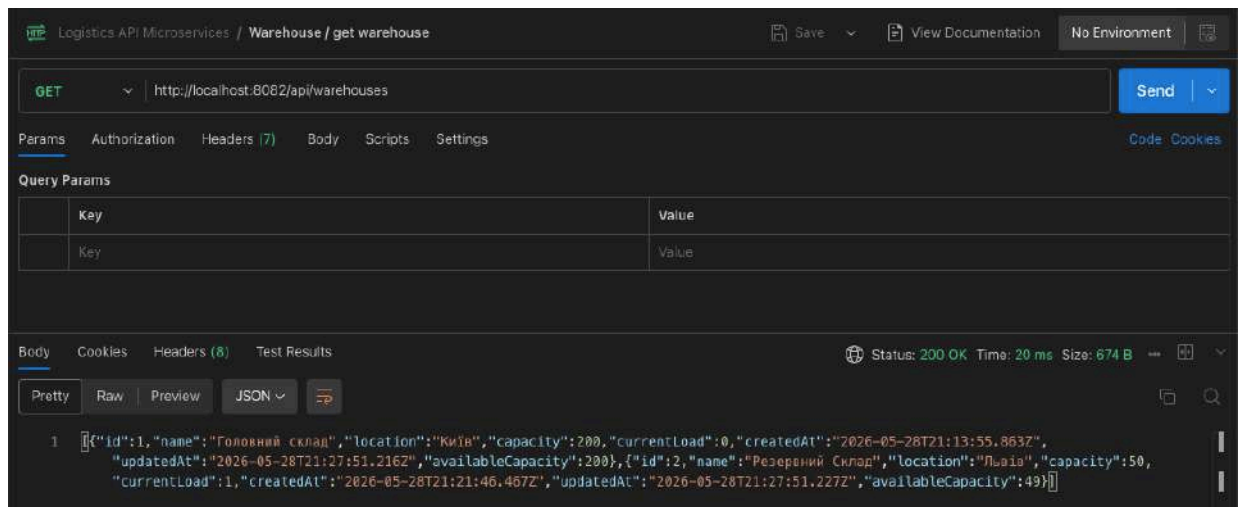


Рис. 13. Отримання списку складів. На Складі 1 місце звільнилося, а на Складі 2 — заповнилося.

3. Імітація недоступності одного з сервісів

Для цього порт 8082 із service-warehouse був зупинений. Перевіримо живучість service-shipment.

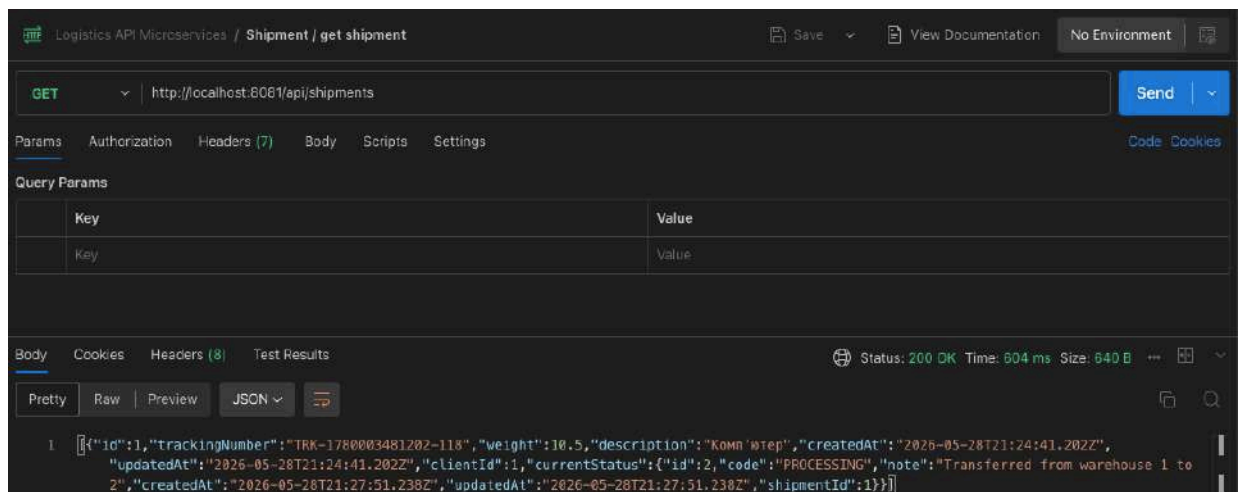


Рис. 14. Отримання списку посилок. Сервіс посилок чудово працює і віддає дані, незважаючи на те, що склади «лежать».

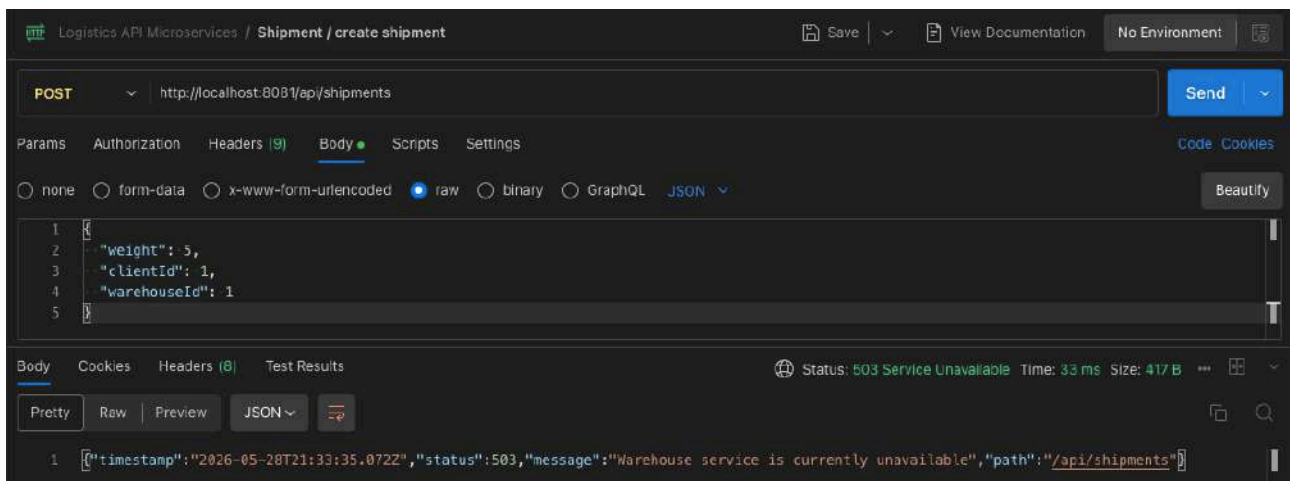


Рис. 15. Тригер помилки недоступності. Сервіс 8081 намагається достукатися до 8082, бачить, що ніхто не відповідає, перехоплює цю помилку в блоці `catch` і коректно відповідає клієнту.

5. Аналіз:

1. Порівняння моноліту та мікросервісів

У монолітній архітектурі вся бізнес-логіка (робота з клієнтами, посилками, складами та маршрутами) фізично знаходилася в одному застосунку та працювала з єдиною базою даних. Усі виклики між модулями виконувалися в оперативній пам'яті, а цілісність даних гарантувалася на рівні реляційної СУБД (Foreign Keys). Після декомпозиції система перетворилася на набір незалежних мікросервісів, кожен з яких відповідає виключно за свій домен та має власну, ізольовану базу даних. Зв'язки на рівні БД зникли, а взаємодія між компонентами тепер відбувається через мережу за допомогою синхронних REST-викликів (HTTP).

2. Переваги мікросервісного підходу

- Ізоляція збоїв: Як було доведено під час тестування, падіння сервісу складів не призводить до повної зупинки системи. Сервіс посилок продовжує працювати і може обслуговувати ізольовані запити (наприклад, створення нових посилок).
- Незалежне масштабування: Якщо сервіс розрахунку маршрутів (Service-Routing) отримає високе навантаження через складні математичні алгоритми, його можна масштабувати (запустити додаткові копії) незалежно від інших частин системи.
- Автономність розробки: Кожен сервіс може мати власний цикл релізів. Оновлення логіки складів не вимагає перекомпіляції та перезапуску сервісу посилок.

3. Недоліки та нові виклики

- Відсутність єдиних транзакцій: Оскільки бази даних розділені, неможливо виконати один SQL-запит, який гарантовано оновить дані в різних сервісах. Замість цього доводиться реалізовувати складні патерни розподілених транзакцій.
- Мережева затримка: Швидкі виклики функцій всередині коду замінилися на HTTP-запити, що збільшує загальний час обробки клієнтського запиту та створює ризик мережевих таймаутів.
- Складність відлагодження: У разі виникнення помилки відстежити шлях запиту через кілька незалежних мікросервісів значно важче, ніж у єдиному моноліті.

4. Ускладнення розгортання

Перехід на мікросервіси кардинально ускладнює інфраструктуру. Навіть у локальному середовищі замість запуску одного процесу `node app.js` та однієї БД довелося конфігурувати три окремі порти (8081, 8082, 8083), три окремі змінні середовища (`.env`) та ініціалізувати три бази даних. У реальному середовищі такий підхід вимагає використання технологій контейнеризації та оркестрації. Також виникає необхідність у додаткових інфраструктурних вузлах: API Gateway (для єдиної точки входу клієнтів), Service Registry (для динамічного пошуку сервісів) та системах централізованого логування, щоб збирати логи з усіх сервісів в одне місце.

Висновок:

У ході виконання роботи було успішно здійснено декомпозицію монолітної системи на незалежні мікросервіси з використанням Node.js та Express. Головним досягненням стало виділення трьох автономних доменів (Посилки, Склади, Маршрути), кожен з яких отримав власну ізольовану базу даних PostgreSQL. Жорсткі зв'язки між сутностями на рівні СУБД були замінені на мережеву взаємодію через REST-виклики (HTTP).

Під час тестування розроблена система продемонструвала високу відмовостійкість: підтверджено, що недоступність одного з модулів не призводить до повної зупинки інших. Також було успішно вирішено проблему розподілених транзакцій за допомогою патерну оркестрації, що дозволило зберегти консистентність даних під час складних міжсервісних операцій. У результаті створено масштабовану, гнучку та надійну архітектуру, яка готова до подальшого розвитку та розгортання в ізольованих контейнерах.